Syntax of first order logic

Professor Cian Dorr

4th October 2022

New York University

First-order languages with definite descriptions

In our first-order languages,

Analogous to the reasons of convenience that motivate allowing function symbols, for our purposes it will be convenient to consider first order languages with one extra innovation: a *definite description* operator ι. For a given first-order signature Σ, the *definite description language* of Σ, $\mathcal{L}_\iota(\Sigma)$ (a.k.a.)

We'll introduce a new formation rule:

$$\frac{P \in \mathcal{L}_\iota(\Sigma) \qquad t \in \text{Terms}(\Sigma) \qquad v \in \text{Var}}{\iota v(t)\ P \in \mathcal{L}_\iota(\Sigma)}$$

So for example, with the signature Arith, the following is now a term:

$$\iota y(0)(y \times y = x)$$

We read it as 'the unique y such that $y \times y = z$ if there is one, otherwise 0'.

First-order languages with definite descriptions

Once we have set up our logic for this extended sort of language, the following will turn out to be theorems:

$$\iota y(0)(y \times y = x) = z \to z \times z = x$$
$$\iota y(0)(y \times y = x) = z \to z' \times z' = x \to z' = z$$
$$\neg\exists y(y \times y = x) \to \iota y(0)(y \times y = x) = 0$$
$$z \times z = x \wedge z' \times z' = x \wedge \neg(z = z') \to \iota y(0)(y \times y = x) = 0$$

Given such theorems, definite descriptions will turn out to be dispensible in a certain sense: every formula with them is equivalent (in a sense we'll explain) to one without them. Nevertheless they are useful to have around.

Defining terms and formulae with definite descriptions

Introducing definite descriptions introduces a new challenge to getting the definitions right. Before, we defined the set of formulae in terms of the set of atomic formulae, and the set of atomic formulae in terms of the set of terms; but now, what the terms are also depends on what the formulae are! So it looks like our definition is going in a circle.

Here's one way around that obstacle. Let's say that a *pre-term* of Σ is any string that's either a variable, an individual constant of Σ, of the form $f($*s* for some string *s*, or of the form ι*s* for some string *s*.

Note that it'll turn out that all terms are pre-terms and no formulae are. So we can define the set of *expressions* of $\mathcal{L}_\iota(\Sigma)$ and then single out the terms as expressions that are pre-terms, and the formulae as expressions that aren't pre-terms.

Definition

The set of expressions of $\mathcal{L}_\iota(\Sigma)$ is the smallest set X such that:

1. Every variable, individual constant, and sentential constant of Σ is in X.
2. Whenever f is an n-ary function symbol of Σ and $s_1, \ldots, s_n$ are pre-terms in X, $f(s_1, \cdots, s_n) \in X$.
3. Whenever R is an n-ary predicate of Σ or $R = =$ and $n = 2$ and $s_1, \ldots, s_n$ are pre-terms in X, $R(s_1, \cdots, s_n) \in X$.
4. Whenever $s \in X$ is not a pre-term, $\neg s \in X$.
5. Whenever $s, t \in X$ are not pre-terms, $(s \wedge t)$, $(s \vee t)$, and $(s \rightarrow t)$ are in X.
6. Whenever $s \in X$ is not a pre-term and $v \in \mathrm{Var}$ $\forall v\ s$ and $\exists v\ s$ are in X.
7. Whenever $t \in X$ is a pre-term, $s \in X$ is not a pre-term, and $v \in \mathrm{Var}$, $\iota v(t)\ s \in X$.

What's going on here is that we are defining the set of expressions as the closure of the set of variables, individual constants, and sentential constants under a family of *partial* functions